CoEP: LLM-Synthesized Repair Tools with Amplified Oracles for Programming Assignments

ANONYMOUS AUTHOR(S)

We present CoEP, which repairs students' programming assignments with a repair tool that a large language model (LLM) synthesizes once per assignment, rather than with a patch the LLM writes for every submission. Recently, LLM-based program repair has substantially improved the share of student submissions that can be repaired automatically. However, these techniques query the model once per buggy submission, so their spending grows with enrollment even though many submissions need "recurring repairs"—edits that other submissions to the same assignment also need—and they accept any patch that passes the instructor's few test cases. This paper aims to move the model's effort from the submission to the assignment without sacrificing coverage or correctness. One major challenge is that whatever the LLM synthesizes must be trusted without trusting the LLM, and the instructor's tests are too thin to arbitrate a repairer that searches many candidates. To address this, CoEP selects worked pairs by clustering the class's edit scripts, synthesizes misconception-named rewrite rules in a counterexample-guided loop, amplifies the instructor's tests with inputs from a synthesized generator labelled by the class's correct programs, repairs every submission locally by composing the rules and transplanting reference hunks, and learns new rules from the few submissions it escalates to a per-submission query. Every repair CoEP returns passes the amplified tests, whatever the LLM writes; a repair composed of j rule applications changes at most j positions of the student's program; and the chance that its search accepts a patch departing from the class on a fraction ε of inputs decays exponentially in the number of amplified tests. Experimental results on 2,970 buggy programs from 11 assignments show that CoEP repairs 99.4% of them, compared with 98.9% for per-submission LLM repair and 85.9% for reference-guided repair, at 21.2 and 27.3 times lower model spend on two LLMs. An ablation attributes the saving to the local-first architecture, correctness on held-out tests (from 90.4% to 98.9% and 98.6% on a first-year course) to the synthesized oracle, and smaller patches to the synthesized rules.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**; *General programming languages*; • **Social and professional topics** → *Computing education.*

Additional Key Words and Phrases: automated program repair, programming assignments, large language models, program synthesis

1 Introduction

Automated program repair (APR) for programming assignments [1, 9, 11, 36, 44] automatically fixes students' incorrect submissions, so that every student in a course receives immediate and personalized feedback [42]. Given an assignment's problem description, its test cases, and a buggy submission, the technique produces a fixed program that passes every test case, from which a hint or a patch is presented to the student. In practice, hundreds of students attempt the same assignment in a course, and online judges accumulate thousands of submissions to the same problem [24]. In particular, the submissions that already pass every test case form a pool of reference programs on which repair can draw.

Two lines of techniques have been developed for this task. Reference-guided techniques [1, 9, 11, 36] align a buggy program with a structurally matching reference program and transplant the regions in which they differ; they need no model and repair each submission at no marginal cost. LLM-based techniques [7, 14, 22, 23, 38, 40, 41, 44] instead query a large language model (LLM) with the buggy program, which removes the need for a matching reference and extends repair to harder problems. As a result, the share of submissions that can be repaired automatically has seen marked improvement [22, 44].

However, our observation is that per-submission repair re-derives the same repairs over and over. In this work, we define a repair as *recurring* if another buggy submission to the same assignment

needs the identical edit to reach its nearest reference program. As shown in Section 3.2, 36.6% of 2,970 buggy programs from 11 assignments need a recurring repair, and 49.9% on first-year course assignments; this is a lower bound, since two submissions that share one misconception rarely share an entire edit. Yet the LLM-based repairer PyDex [44] pays one query for every submission, $292.77 over these programs on `claude-sonnet-5`, and prompt caching removes only 10.2% of that spend. Reference-guided repair avoids the spend, but Refactory [11] repairs only 54.5% of the buggy programs on online-judge problems rated 1200 and above, where the pool seldom holds a structurally matching program. Consequently, a course must choose between coverage and a model spend that grows with enrollment and recurs every term.

In this paper, we present CoEP (Cost-Effective Program repair), which has an LLM synthesize an assignment-specific repair tool once per assignment, rather than a patch once per submission. Our starting point is the observation of Singh et al. [31] that a small *error model*—a set of rewrite rules capturing the mistakes a class makes—corrects most incorrect attempts at an assignment; their error models, however, are written by hand for each assignment. The key insight of CoEP is that the assignment-specific parts of a repair tool can themselves be synthesized, while the search that uses them stays fixed. A recurring repair is a transformation that can be written down once as an abstract syntax tree (AST) rewrite rule named after the misconception it repairs, and what counts as a correct repair can be sharpened once per assignment by a generator of test inputs that the class's correct programs label. A builder LLM writes both from the problem description and a handful of worked (buggy, reference) pairs. The deployed tool then repairs most submissions with no model at all, and only what it cannot repair is sent to a conventional per-submission query.

However, a key challenge is that recurring repairs rarely recur verbatim. As reported in Section 3.2, the 2,970 buggy programs need 2,228 distinct edit scripts. Even on cAPS lOCK, where a single rule repairs 105 of the 560 buggy programs, the largest group of programs sharing an identical edit script holds only 31. Thus, a builder must generalize each repair beyond the program it was shown, and the code it writes must be trusted without trusting its own claims about that code. A second obstacle is that the instructor's tests are thin, 7.4 per assignment on the first-year course, so a candidate that passes them may still be wrong, and a tool that searches many candidates is precisely the repairer most likely to find one that fits the tests rather than the problem.

To address these challenges, CoEP treats the LLM as an untrusted synthesizer and validates everything it produces by execution. It performs four stages: (1) *Select*, which pairs each buggy program with its minimum-edit reference program and clusters the class by the resulting edit scripts, choosing one worked pair per cluster; (2) *Synthesize*, a counterexample-guided loop in which the builder writes misconception-named rewrite rules and is shown the worked pairs they still fail to repair, and in which it also writes an input generator whose inputs the class's correct programs label by majority vote, amplifying the instructor's tests T into a suite T^+; (3) *Repair*, which repairs every submission by a breadth-first search over compositions of the frozen rules and a hunk-level reference transplant, escalating to a per-submission query only on abstentions and oversized patches; and (4) *Learn*, which distills the escalated repairs into new rules. We prove that every repair CoEP returns passes T^+ whatever the builder writes, that a rule repair composed of j rule applications changes at most j positions of the program, since CoEP enforces a single-site contract on every rule, and that a local search over N candidates accepts a patch departing from the class on more than a fraction ε of generated inputs with probability at most $N(1-\varepsilon)^n$ for n amplified tests, so the broader the search, the more the oracle matters; we also show that the rule search returns a shallowest composition, that escalation never loses coverage or enlarges a patch, and that repair terminates (Section 4.7).

Experimental results demonstrate that CoEP markedly reduces the cost of LLM-based repair without sacrificing coverage or correctness. We evaluated CoEP with two builders, `claude-sonnet-5`

and `gpt-5.4-mini`, on 2,970 buggy programs from 11 assignments, five from a first-year course [11] and six Codeforces problems rated 800 to 2000 [24], against Refactory [11] and PyDex [44]. CoEP repaired 99.4% of the programs, where Refactory repaired 85.9% and PyDex 98.9%, with patches 31% smaller than PyDex's, and spent $13.83 and $1.04 on the two models, 21.2 and 27.3 times less than PyDex. An ablation attributes these results: without the synthesized rules, CoEP still spends 79 to 173 times less than PyDex, so the saving comes from the local-first architecture; without the synthesized oracle, its held-out repair rate on the first-year course falls from 98.9% and 98.6% to 90.4%; and the synthesized rules raise the share of programs repaired with a patch of RPS at most 0.25 by 5.7 and 1.8 points. A tool built from the first 10% of a class repaired 99.3% and 98.9% of the last quarter's buggy programs on the two corpora.

Contributions. We summarize our contributions below:

- **Formulation:** We formulate repair for programming assignments as the synthesis of the two assignment-specific parts of a repair tool—a set of AST rewrite rules, and an acceptance suite amplified by an input generator—with an LLM as an untrusted synthesizer of both and a fixed search that combines them with a reference transplant, generalizing the hand-written error models of prior work [31] (Sections 3 and 4).
- **Technique and guarantees:** We present CoEP, an algorithm that selects worked pairs by clustering edit scripts, synthesizes misconception-named rules in a counterexample-guided loop, amplifies the tests by differential testing, repairs every submission locally by searching rule compositions and transplanting reference hunks behind a gated fallback, and learns new rules from escalated repairs; we enforce a single-site contract on the synthesized rules and prove a locality theorem that bounds the size of rule repairs, along with soundness relative to T^+, the label soundness of the amplified oracle, a bound on the probability that its search accepts an overfitted patch, and three further properties (Section 4).
- **Evaluation:** We evaluate CoEP against reference-guided and LLM-based repair on 2,970 buggy programs from 11 assignments, including the prevalence of recurring repairs, an ablation that attributes cost, correctness, and patch size to the architecture, the oracle, and the rules, held-out test cases, and splits by submission order. We make CoEP, the 22 synthesized tools, the 10 input generators, every prompt, and every per-submission log publicly available (Data-Availability Statement).

2 Overview

We illustrate CoEP on two assignments of our benchmark, cAPS lOCK and sequential search.

A recurring misconception. cAPS lOCK (Codeforces 131A) asks whether a word was typed with the caps lock on, i.e., whether all its letters, or all but the first, are uppercase. Suppose a student tests the tail of the word with `s[1:].isupper()` at line 4 of Figure 1-(a). The program then fails on a one-letter word such as `"z"`, since the tail is empty and an empty string is not uppercase in Python. 105 of the 560 buggy programs submitted to this problem share this misconception. Per-submission LLM repair queries the model 105 times to rediscover the same fix, and a reference transplant must import a length guard together with the structure around it. Nor do the 108 programs share one edit: they differ elsewhere, so their edits to the nearest correct program differ too.

Synthesizing the tool. CoEP builds the tool before the first repair. Select pairs each of the 560 buggy programs with the correct program it is fewest edits from and clusters the programs by these edit scripts, yielding 353 worked pairs. Synthesize shows them to a builder LLM, which answers with a Python module of rewrite rules; CoEP runs the rules on the worked pairs and returns the pairs they still fail to repair, for three rounds. On cAPS lOCK, `claude-sonnet-5` wrote 10 rules for $1.23. Figure 1-(b) shows one, `slice_case_check_to_equality`: it locates a case test on a slice

```
s=input()
c=s[0]
if c.islower():
    if s[1:].isupper():
        rs=s.swapcase()
        print(rs)
    else:
        print(s)
elif s.isupper():
    es=s.swapcase()
    print(es)
else:
    print(s)
```

(a) A buggy submission. The rule rewrites line 4 to `if s[1:].upper() == s[1:]`: (RPS = 0.118).

```
def _rule_slice_case_to_equality(source):
    """slice.isupper()/islower() fails on an empty slice; the
    equality form also fixes the isupper/islower mix-up."""
    tree = ast.parse(source)
    out = []
    for node in ast.walk(tree):
        if (isinstance(node, ast.Call) and not node.args
                and not node.keywords
                and isinstance(node.func, ast.Attribute)
                and node.func.attr in ("isupper", "islower")
                and isinstance(node.func.value, ast.Subscript)):
            vs, ve = _span(source, node.func.value)
            value_text = source[vs:ve]
            new_text = "%s.upper() == %s" % (value_text, value_text)
            candidate = _splice(source, node, new_text)
            if candidate != source:
                out.append(candidate)
    return out
```

(b) The rule `slice_case_check_to_equality` written by `claude-sonnet-5` (abridged).

Fig. 1. A recurring misconception on cAPS lOCK (Codeforces 131A) and the synthesized rule that repairs it.

```
def search(x, seq):
  for i in range(1, len(seq)+1):
    if x < seq[i-1]:
      return i-1
    elif seq[i-1]<x<seq[i]:
      return i
  return i
```

(a) A buggy submission.

```
def search(x, seq):
  for i in range(len(seq)-1):
    if seq[i] < x <= seq[i+1]:
      return i+1
    elif x <= seq[0]:
      return 0
  return len(seq)
```

(b) Accepted by T; `search(-15, [-6])` returns 1 (RPS = 0.47).

```
def search(x, seq):
  if not seq:
    return 0
  elif x <= seq[0]:
    return 0
  else:
    for i in range(len(seq)-1):
      if seq[i] < x <= seq[i+1]:
        return i+1
    return len(seq)
```

(c) Accepted by T^+ (RPS = 0.84).

Fig. 2. Patches for one submission to sequential search (R1) under the instructor's test cases T and the amplified suite T^+.

and splices in an equality that also holds for the empty slice. The rule was written from one worked pair but is stated over the AST, so it applies wherever the pattern occurs.

Repairing locally. The frozen tool repairs each submission by a breadth-first search over compositions of its rules, validating every candidate by running the tests, and transplants line hunks from the reference programs when no composition passes. On cAPS lOCK, `slice_case_check_to_equality` repaired 104 programs, the tool's rules 145 in all, and the transplant the remaining 415, without a model call; the repair of Figure 1-(a) changes one line, at a relative patch size (RPS) of 0.118.

When the tests are thin. Sequential search (R1) asks for the position at which x would be inserted into a sorted sequence, and its instructor ships 11 test cases, none with a one-element sequence. For the buggy program of Figure 2-(a), the transplant finds the patch of Figure 2-(b), which passes all 11 tests but returns 1 for `search(-15, [-6])`: its guard `x <= seq[0]` sits inside a loop that never runs on a one-element sequence. The builder's input generator draws sorted sequences, one-element ones included, and the class's correct programs label each input by majority. 100 such inputs join the 11 tests in T^+, and the patch of Figure 2-(b) fails 10 of them. The search therefore continues and accepts the patch of Figure 2-(c), transplanted from a reference program with an explicit guard, which also passes the held-out tests of Section 5.6. The price is a larger patch, at RPS 0.84 against 0.47.

Algorithm 1 Repairing an Assignment

Input: Description (d), tests (T), buggy programs (B), reference programs (R), local repair tool (L).
Output: Fixed programs (F).

```
1:  procedure RepairAssignment(d, T, B, R, L)
2:      F ← ∅
3:      for each b ∈ B do
4:          f ← fail
5:          if L ≠ ∅ then
6:              f ← LocalRepair(L, b, R, T)
7:          if f = fail or RPS(b, f) > η_rps then
8:              for each c ∈ Generate(d, T, b, R) do
9:                  if c ⊨ T ∧ RPS(b, c) < RPS(b, f) then f ← c
10:         F ← F ∪ {(b, f)}
11:     return F
```

Escalation and learning. When both local strategies abstain, or the local patch is at least as large as the program, CoEP issues one query that attaches the minimum-edit reference program (12.6% of the submissions), and the Learn stage generalizes such repairs into new rules. Whichever path a repair takes, it passes T^+ (Section 4.7).

3 Preliminaries

3.1 Program Repair for Programming Assignments

The main aim of program repair for programming assignments is to turn each buggy submission into a fixed program that passes the assignment's test cases. To do so, a repairer iteratively generates, validates, and selects candidate programs for every submission. We define an assignment as a quadruple (d, T, B, R), where d represents the natural-language problem description, $T = \{t_1, \ldots, t_n\}$ denotes the instructor's test cases, B indicates the buggy programs, and R the reference programs. Writing $p \models T$ when a program p passes every test case in T, the submissions divide into $B = \{p \mid p \not\models T\}$ and $R = \{p \mid p \models T\}$. To repair $b \in B$ is to produce a fixed program f with $f \models T$; a candidate that passes only some test cases is not a repair.

Algorithm 1 describes a generic algorithm that repairs an assignment. It takes five inputs: the description (d), the test cases (T), the buggy programs (B), the reference programs (R), and a local repair tool (L). For now, let us set aside the last input, L, and the associated function, LocalRepair; their roles will be explained in Section 4. Existing techniques run Algorithm 1 without a tool (i.e., $L = \emptyset$). At line 2, the algorithm initializes the set F of fixed programs as an empty set. It then repeats the loop at lines 3–10 for each buggy program b. At line 8, the Generate function produces candidate programs from b, and line 9 keeps a candidate c that passes every test case and is smaller than the current patch, with $\mathrm{RPS}(b, \mathit{fail}) = \infty$, so the smallest passing candidate survives, as in PyDex [44]. Line 7 also escalates an oversized local patch, $\mathrm{RPS}(b, f) > \eta_{rps}$, for reasons Section 4.5 gives. Finally, Algorithm 1 returns the set F of pairs of a buggy program and its fixed program, where $f = \mathit{fail}$ marks a program that was not repaired.

Existing techniques for programming assignments differ in how they implement the Generate function at line 8. Reference-guided techniques align b with a structurally matching reference program $r \in R$ and transplant the regions in which they differ: CLARA [9] and Sarfgen [36] align b with its nearest reference program, Verifix [1] backs the alignment with formal verification, and Refactory [11] refactors the reference programs to widen the range of alignable programs. They run offline at no marginal cost, but Generate returns no candidate when R holds no program whose structure matches b. LLM-based techniques implement Generate as a model query: PyDex [44] prompts the model with d, b, and three (buggy, correct) exemplars retrieved from the class and samples ten candidates, while others add conversation [38], tutoring [40], fine-tuning [14, 41], or fault localization [22, 23]. These techniques repair submissions that no reference matches, but each iteration of the loop at line 3 issues its own query, so the number of queries grows with $|B|$.

Table 1. Recurring repairs and the cost of repairing them one submission at a time. DISTINCT: distinct edit scripts to the minimum-edit reference program. RECURRING: buggy programs whose edit script another buggy program of the same assignment also needs.

Corpus	Assignments	Buggy	Edit scripts			Refactory	PyDex
			DISTINCT	RECURRING		RR (%)	SPEND ($)
CS1	R1–R5	1,783	1,183	890	(49.9%)	90.8	146.27
Codeforces 800–1000	C1–C2	1,000	861	192	(19.2%)	83.1	113.71
Codeforces 1200–2000	C3–C6	187	184	5	(2.7%)	54.5	32.79
Total	11	2,970	2,228	1,087	(36.6%)	85.9	292.77

3.2 Limitation of Per-Submission Repair

Our key observation is that many submissions to one assignment need the same repair, yet Algorithm 1 derives each of them independently. In this work, we newly define a *recurring repair* as a repair whose edit another buggy submission to the same assignment also needs. More precisely, let $\pi(b)$ be the reference program that takes the fewest AST edit operations from b, and $\varepsilon(b)$ the multiset of edit operations that takes b to $\pi(b)$ (both defined in Section 4.3); the repair of b is recurring if $\varepsilon(b) = \varepsilon(b')$ for some other $b' \in B$.

Table 1 shows that 1,087 of the 2,970 buggy programs (36.6%) across the 11 assignments of our benchmark (Section 5.1) need a recurring repair, and 49.9% on the five first-year assignments. Nevertheless, the per-submission repairer PyDex [44] spent $292.77 on `claude-sonnet-5` querying for each of them, so that a single misconception is paid for once per student who makes it. Surprisingly, the share of recurring repairs falls to 2.7% on Codeforces problems rated 1200 to 2000, precisely where the reference-guided Refactory [11] repairs only 54.5% of the programs. This might stem from the larger number of solution strategies that harder problems admit: each buggy program needs an edit of its own, and the pool seldom holds a program of the same structure.

Making each query cheaper does not suffice: prompt caching [2, 20] removed only 10.2% and 17.3% of PyDex's spend, since output tokens dominate it (Section 5.3), and memoizing patches by submission text helps even less, as only 8.6% of the buggy programs are textual duplicates of another.

3.3 Goal

The goal of this paper is to repair every buggy program of an assignment while spending model queries per assignment rather than per submission. Formally, we aim to synthesize a local repair tool L, with a number of builder queries that does not grow with $|B|$, such that the set

$$\{\, b \in B \mid \textsc{LocalRepair}(L, b, R, T) \neq \mathit{fail} \,\}$$

is maximized, where LOCALREPAIR is the hook at line 6 of Algorithm 1, and every repair passes an acceptance suite $T^+ \supseteq T$ strong enough for the search that found it; the remaining programs fall through to GENERATE at line 8. This is far from straightforward: even among recurring repairs, the largest group sharing one edit script holds only 48 programs (R4). Our work thus tackles three key challenges: (1) selecting, from thousands of distinct edits, the worked examples from which a builder can generalize a repair beyond the program it saw, (2) trusting the code a builder writes without trusting its claims about that code, and (3) accepting a repair only against tests strong enough for the breadth of the search that found it.

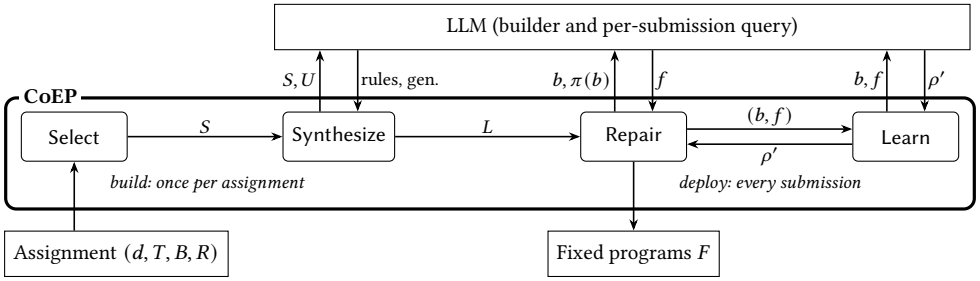


Fig. 3. Overview of CoEP

Algorithm 2 CoEP

Input: Description (d), tests (T), buggy programs (B), reference programs (R), builder LLM (M).
Output: Fixed programs (F), repair tool (L').

procedure CoEP(d, T, B, R, M)
  /* **Step 1. Select** */
  $S \leftarrow \text{Select}(B, R)$ ▷ $(b^\star, \pi(b^\star)) \in S$
  /* **Step 2. Synthesize** */
  $\langle U, j \rangle \leftarrow \langle S, 0 \rangle$ ▷ $\eta_{round} = 3$
  repeat
    $j \leftarrow j + 1$
    $L_j \leftarrow \text{Synthesize}(M, d, T, S, U)$
    $U \leftarrow \{(b, _) \in S \mid \text{Search}(L_j, b, T) = \mathit{fail}\}$
  until $U = \emptyset$ **or** $j = \eta_{round}$
  $L \leftarrow \arg\max_{i \le j} \text{cov}(L_i)$
  $T^+ \leftarrow T \cup \text{Amplify}(M, d, T, R)$
  /* **Step 3. Repair** */
  $F \leftarrow \textsc{RepairAssignment}(d, T^+, B, R, L)$
  /* **Step 4. Learn** */
  $L' \leftarrow L$
  for each $(b, f) \in F$ repaired by GENERATE **do**
    $\rho' \leftarrow \text{Learn}(M, b, f)$
    if $\text{Search}(L \cup \{\rho'\}, b, T^+) \neq \mathit{fail}$ **then** $L' \leftarrow L' \cup \{\rho'\}$
  return F, L'

4 Our Approach

4.1 Architecture

Figure 3 illustrates how CoEP synthesizes and deploys an assignment-specific repair tool while interacting with an LLM. The first two stages run once per assignment and build the tool. Select pairs each buggy program with its minimum-edit reference program and chooses one worked pair per cluster of edit scripts, without querying a model. In Synthesize, a builder LLM turns the worked pairs into a Python module of AST rewrite rules, each named after the misconception it repairs, and CoEP executes the test cases to return the pairs on which the rules still fail, up to a round budget. The builder also writes a generator of test inputs, which CoEP labels by the majority output of the reference programs to amplify T into T^+. The last two stages run on every submission. In Repair, the frozen tool tries its rules and then a hunk-level transplant, accepting a candidate only if it passes T^+, and issues a per-submission query only when both abstain or the local patch is larger than the program it repairs. In Learn, the model generalizes each repair obtained from such a query into a new named rule, which the tool keeps only if the rule reproduces the repair.

The key novelty of CoEP lies in moving the model from the repair of each submission to the construction of the two assignment-specific parts of a repair tool, its rules and the generator behind its oracle, both ordinary code that runs offline and can be audited, while the search that combines them with the reference transplant stays fixed. Section 5.4 measures what each part buys: the local-first architecture accounts for the saving over per-submission repair, the oracle for correctness beyond the instructor's tests, and the rules for small patches.

4.2 CoEP

Algorithm 2 details how CoEP synthesizes a repair tool for an assignment and repairs its submissions. CoEP accepts the assignment (d, T, B, R) and a builder LLM (M), and returns the fixed programs (F) together with the tool. At line 3, the Select function chooses the build set S (Section 4.3). At lines 5–11, CoEP synthesizes the tool in at most η_{round} rounds: each round asks the builder for a module L_j (line 8), collects the worked pairs U it still fails to repair (line 9) for the next round, and the round with the highest coverage is kept (line 11). At line 12, the Amplify function builds the acceptance suite T^+ (Section 4.4). At line 14, CoEP repairs every buggy program by calling Algorithm 1 with the frozen tool L and T^+, and at lines 16–19, it learns new rules from the repairs obtained from a model query. In our experiments, η_{round} was set to 3, and every other constant below is fixed across all assignments.

4.3 Select

The goal of the Select stage is to choose a set of worked pairs from which the builder can generalize the recurring repairs of the class (line 3). This stage is local and issues no model query. To pair programs, CoEP compares their ASTs. Let $\sigma(p)$ be the sequence of node types of p's AST read in breadth-first order, split into one sequence $\sigma_i(p)$ per top-level statement. With lev the Levenshtein distance between two such sequences, we define the distance between a buggy program b and a reference program r as the cost of the cheapest one-to-one matching $\mathcal{M}$ between their sequences:

$$\delta(b, r) = \min_{\mathcal{M}} \Big[\sum_{(i,j)\in\mathcal{M}} \mathrm{lev}(\sigma_i(b), \sigma_j(r)) + \sum_{i\notin\mathcal{M}} |\sigma_i(b)| + \sum_{j\notin\mathcal{M}} |\sigma_j(r)| \Big]$$

where an unmatched sequence costs its full length, so a missing or surplus definition costs what it is. Each buggy program then takes as partner the reference program it is fewest edits from, $\pi(b) = \arg\min_{r\in R} \delta(b, r)$, and $\varepsilon(b)$ denotes the multiset of edit operations that takes b to $\pi(b)$. Intuitively, $\pi(b)$ is the reference program that solves the problem the way b tries to, rather than the one that merely looks closest. When the node-type alignment yields no edit, $\varepsilon(b)$ is refined to an alignment over identifiers and literals, since a wrong variable or constant is invisible to node types alone.

CoEP then partitions B into clusters $C_1 \uplus \cdots \uplus C_k$ by average-linkage agglomerative clustering over the cosine distance between TF-IDF vectors of the edit scripts $\varepsilon(b)$, choosing k to maximize the mean silhouette coefficient [29]. We selected edit scripts as the clustering feature because a rule written from one member pays off only on the members needing the same edit; clustering on the programs' own AST paths instead filled the largest clusters with programs that merely look alike. SemCluster [25] clusters programs by semantic features because syntax over-partitions them, but the argument does not carry over to edits: an edit script cancels the syntax b shares with $\pi(b)$, and the consumer of a cluster, a rewrite rule, matches syntax. Each cluster contributes its modal member,

$$b_i^\star = \arg\max_{b\in C_i} \big|\{b' \in C_i \mid \varepsilon(b') = \varepsilon(b)\}\big|,$$

whose edit the cluster most agrees on. The medoid, usually a tangle of several faults, serves only for a cluster whose edits never repeat. The build set is $S = \{(b_i^\star, \pi(b_i^\star))\}_{i=1}^{k}$, ordered by increasing δ, so a weak builder is shown each buggy program with a reference program that repairs it and need only generalize the transformation. Empirically, Select chose 1,524 worked pairs over our 11 assignments, from 2 to 353 per assignment.

4.4 Synthesize

The goal of the Synthesize stage is to turn the build set into an assignment-specific repair tool and the suite that accepts its repairs (lines 5–12). At line 8, the Synthesize function prompts the builder M with the description d, up to $\eta_{test} = 8$ test cases of T as examples, and the worked pairs of S. The builder answers with a Python module L_j of AST rewrite rules, each named after the misconception it targets and tagged with the failure symptoms it addresses (e.g., a wrong answer or a `TypeError`). Formally, a rule is a candidate generator ρ that maps a program p to a set $\rho(p)$ of candidate programs. Figure 1-(b) shows one such rule. The contract on a rule is strict, and CoEP enforces it. A rule locates its edit on the AST and splices it in as a source-span replacement, rather than re-rendering the module; it proposes candidates rather than claiming a fix; and each candidate $c \in \rho(p)$ is a *single-site* rewrite of p: $p = C[t]$ and $c = C[t']$ for a one-hole context C whose hole is one subtree, or one run of a statement or expression list. Since rules are Python over the `ast` module, the last clause cannot be checked statically, so CoEP checks it dynamically, discarding before validation every candidate that differs from p at more than one maximal AST position; before we enforced it, 9.8% of the builders' rule repairs broke it. A rule that cannot express a targeted edit, raises an exception, or runs beyond an $\eta_{alarm} = 2$-second alarm yields $\rho(p) = \emptyset$, so its submission falls through to the next strategy.

Since a module written in one shot can contain its own defects, CoEP closes the loop on the build set itself, in the manner of counterexample-guided inductive synthesis [33] and of self-refinement from execution feedback [16, 30]: the worked pairs the rules still fail are the counterexamples. We define the coverage of a module as the fraction of the build set whose buggy half its rules repair:

$$\mathrm{cov}(L_j) = \frac{|\{(b, _) \in S \mid \mathsf{Search}(L_j, b, T) \neq \mathit{fail}\}|}{|S|}$$

where Search is the rule search of Section 4.5. Coverage is measured by executing T, never from the module's own claim, and it excludes the reference transplant, so a round is scored on the rules the builder wrote rather than on the substrate beneath them. While $\mathrm{cov}(L_j) < 1$, line 9 returns the still-failing pairs U to the builder for revision. The frozen tool L is the round with the highest coverage (line 11), ties going to the earliest. Intuitively, the refinement lets a builder repair its own rules from concrete failures, while the arg max ensures that a revision never replaces a better module.

Amplified acceptance. The instructor's test cases are few, 7.4 per assignment on R1–R5, and every strategy of the Repair stage accepts any candidate $c \models T$, so a thin T admits patches that fit the tests rather than the problem. At line 12, the Amplify function therefore enlarges the acceptance suite once per assignment, by differential testing against the reference programs. The builder M is prompted with d, T, and the instructor's reference solution, and writes a generator of random valid inputs, which CoEP validates as it validates rules: a round fails if the generator raises, returns a malformed input, or yields inputs of which the reference solution rejects more than 10%, and the failure is returned to the builder for at most η_{round} rounds. CoEP then draws $\eta_{draw} = 300$ fresh inputs and labels each with the majority output of $\eta_{vote} = 40$ programs sampled from R together with the reference solution, keeping an input only if at least $\theta = 90\%$ of them agree, so an input on which correct programs disagree, as they may outside the specification, never becomes a test. With V these $|V| = 41$ voters and y_x their majority output on x, the first $\eta_{amp} = 100$ kept inputs form

$$T_{amp} = \{\, (x, y_x) \mid |\{v \in V \mid v(x) = y_x\}| \geq \theta\,|V| \,\},$$

and $T^+ = T \cup T_{amp}$. Since Amplify runs once the rules are frozen, no rule depends on T_{amp}, which Theorem 4.3 relies on. T^+ replaces T in every acceptance check of the Repair and Learn stages, whereas the coverage of the rules is still measured on T. This is differential testing [17] with the

class as the oracle, in the spirit of test generation against overfitted patches [32, 39]; it adds one builder query per assignment and none per submission.

4.5 Repair

The goal of the Repair stage is to repair each buggy program b with the frozen tool, and to spend a model query only where the tool cannot (line 14). CoEP implements the LocalRepair hook of Algorithm 1 (line 6) with two strategies in increasing order of cost, and the Generate function (line 8) with a conventional model query; whichever first yields a candidate c with $c \models T^+$ supplies f.

Rule-based repair. The Search function composes the rules by a breadth-first search over layers of rewritten programs, $\Lambda_0 = \{b\}$ and $\Lambda_{j+1} = (\bigcup_{p\in\Lambda_j} \bigcup_{\rho\in L} \rho(p)) \setminus \bigcup_{i\le j} \Lambda_i$, where the failing-test signature of b orders the rules whose declared symptoms match it. Every candidate in Λ_{j+1} is validated before Λ_{j+2} is enumerated, so the returned repair is the shallowest candidate c with $c \models T^+$. For the program of Figure 1-(a), Λ_1 already holds the splice of Figure 1-(b) at line 4, which passes, so the search returns it at depth 1. The search validates at most $\eta_{search} = 1{,}000$ candidates, a count rather than a time, so the repair it returns does not depend on the machine; it does not rewrite a candidate that ran past its time limit further, since such rewrites inherit the loop that hangs it. Alongside the builder's rules, every tool carries a handful of always-on rules for the slips any class makes, such as flipped comparisons and off-by-one range and index shifts, which invert classical mutation operators [12, 13, 15]. Every repair the rules produce carries the ordered names of the rules behind it.

Reference-based repair. Most student bugs need a structural edit that no token-level rule expresses, and R supplies it. For each of the $\eta_{ref} = 8$ reference programs nearest to b under δ, CoEP computes the set $H(b, r)$ of line hunks in which b and r differ, both for r as written and for r with its variables renamed to b's (parameters by position, locals by first binding). Let $b \oplus h$ denote b with the hunks $h \subseteq H(b, r)$ applied. CoEP validates, across them, the $\eta_{cand} = 300$ subsets h with $1 \le |h| \le 3$ that change the fewest lines, and returns $\arg\min_{h\in P} \mathrm{RPS}(b, b \oplus h)$ over the first $\eta_{pass} = 5$ passing subsets P, abstaining when none passes. Here, the relative patch size $\mathrm{RPS}(b, f)$ is the tree-edit distance [45] between the ASTs of b and f divided by the size of b's AST [11]. This is reference reuse in the lineage of CLARA, Sarfgen, and Refactory [9, 11, 36], but at the granularity of line hunks rather than whole blocks: a hunk remains applicable when the student's strategy differs from the reference program's elsewhere.

LLM-based repair. When both local strategies abstain, CoEP attaches the minimum-edit reference program from the Select stage and issues a single query $\mathrm{LLM}(d, T, b, \pi(b))$, so that the model repairs b toward the student's own strategy. CoEP also escalates when the local strategies succeed with $\mathrm{RPS}(b, f_{loc}) > \eta_{rps} = 1$, i.e., with a passing patch at least as large as the program it repairs. The reason is that the acceptance test, $c \models T^+$, says nothing about patch size: a transplant can pass T^+ by replacing most of a student's program, whereas a rule must abstain whenever it cannot express a targeted edit. CoEP keeps

$$\arg\min_{f\in\{f_{loc},\,\mathrm{LLM}(d,T,b,\pi(b))\},\ f\models T^+} \mathrm{RPS}(b, f),$$

so an escalation can cost a query but never coverage or patch size. The threshold is a definition rather than a fitted value.

4.6 Learn

The goal of the Learn stage is to absorb the repairs the tool could not make into the tool itself (lines 16–19). For each pair (b, f) that Algorithm 1 repaired with a model query, the Learn function

asks M to generalize the edit from b to f into a new named rule ρ'. CoEP keeps ρ' only if the tool extended with it repairs b by itself, i.e., $\text{Search}(L \cup \{\rho'\}, b, T^+) \neq \mathit{fail}$. Learned rules are written to a staging copy L' that the current pass never consults, so no submission's outcome depends on which submissions preceded it, and L' replaces L only when the tool is frozen again. Since Learn queries the model only for submissions that already escalated, it adds no query for a submission the tool repairs.

The deployed tool itself never queries a model. Its local strategies run on a CPU in about four seconds per submission (Section 5.3), so once the tool is built, student code leaves the institution only on escalation, and a defective rule degrades into abstention rather than a failing repair, because every candidate is validated against T^+ before it is returned.

4.7 Properties

We now state what CoEP guarantees whatever the builder writes. Throughout, $c \models T$ means that the candidate c passes every test case of T within its time limit, so a candidate that crashes or times out does not pass; CoEP_L runs Algorithm 1 without Generate, and CoEP_LF with Generate and the gate of Section 4.5.

Trust boundary. The builder's rules and generator are untrusted code. A rule runs in the search process under the η_{alarm} alarm, which discards its output if it raises or runs over; every candidate and every generated input runs in a fresh subprocess under a per-test time limit; and the fallback query is retried at most five times, each under the client's request timeout. We assume that a rule computes its candidates without modifying the search process itself, e.g., by patching the test runner: the builder is untrusted to be correct, not assumed to be adversarial. Isolating rules in subprocesses as well would discharge this assumption at the cost of one process per rule call.

Proposition 4.1 (Soundness relative to T^+). *For every builder M and every $b \in B$, if CoEP returns $f \neq \mathit{fail}$ for b, then $f \models T^+$, and hence $f \models T$.*

Proof. Each path into F validates its candidate against T^+ before returning it: Search and the transplant return only passing candidates, a candidate of Generate is kept only if $c \models T^+$ (line 9 of Algorithm 1, called with T^+), the gate keeps an arg min over passing candidates, and learned rules enter only through Search. What the builder and the model write only proposes candidates and tests. Finally, $T \subseteq T^+$. □

Proposition 4.1 implies that amplification can only shrink the set of accepted candidates; Section 5.6 measures the exchange at 0.3 to 0.5 points of repair rate on T for 8.2 to 8.5 points on held-out tests. What the amplified tests are worth depends on their labels, which the next proposition bounds.

Proposition 4.2 (Label soundness). *Let $(x, y) \in T_{amp}$. If fewer than $\lceil \theta|V| \rceil$ voters produce one and the same incorrect output on x, then y is the correct output on x.*

Proof. By construction, y is the output of at least $\lceil \theta|V| \rceil$ voters. If y were incorrect, these voters would produce one and the same incorrect output on x, contrary to the hypothesis. □

With $|V| = 41$, a wrong label therefore needs 37 voters, sampled from programs that pass T, to make the same mistake on the same input: a misconception shared by most of the class, which is precisely what no test labelled by the class can expose (Section 5.7). In our ten amplified suites, every label had at least 38 of the 41 votes, 708 of the 1,000 were unanimous, and the instructor's reference solution agreed with every label.

Let D be the distribution of the inputs that Amplify keeps, and for a program c, let $\text{err}(c) = \Pr_{x \sim D}[c(x) \neq y_x]$ be the probability that c departs from the class's label. Let $C_k(b)$ be the candidates of the transplant for b together with those of the rule layers $\Lambda_1, \ldots, \Lambda_k$.

Theorem 4.3 (Overfitting bound). *Suppose the n inputs of T_{amp} are drawn independently from D. For every b, every k, and every $\varepsilon > 0$, the probability that the local strategies return a repair $c \in C_k(b)$ with $\mathrm{err}(c) > \varepsilon$ is at most $|C_k(b)|\,(1-\varepsilon)^n$.*

Proof. $C_k(b)$ does not depend on T_{amp}: the rules are frozen before Amplify runs, a rule maps a program to candidates without seeing the tests, and the transplant draws its hunks from R alone; the tests only order and filter what is enumerated. Fix $c \in C_k(b)$ with $\mathrm{err}(c) > \varepsilon$. Each input of T_{amp} exposes c independently with probability $\mathrm{err}(c)$, so c passes all n of them with probability $(1-\mathrm{err}(c))^n < (1-\varepsilon)^n$. A returned repair passes $T^+ \supseteq T_{amp}$ (Proposition 4.1), and a union bound over the at most $|C_k(b)|$ such candidates gives the claim. □

The bound quantifies why the broader the search, the more the oracle matters: $n \geq (\ln|C_k(b)| + \ln(1/\delta))/\varepsilon$ keeps it below δ. With $n = 100$, every local repair is drawn from at most $\eta_{search} + \eta_{cand} =$ 1,300 validated candidates, so it departs from the class on more than 10% of generated inputs with probability below 3.5% (2.7% for the median repair on R1–R5), whereas a single query needs the oracle far less. The patch of Figure 2-(b), for instance, fails 10 of R1's 100 amplified inputs, and a patch departing on 10% of D passes 100 independent draws with probability $0.9^{100} < 3 \cdot 10^{-5}$. Section 5.6 observes the interaction the bound predicts: amplification raised CoEP's held-out repair rate by 8.5 and 8.2 points but did not help $\mathrm{LLM}_{\mathrm{Direct}}$, and what it gave PyDex grew with the number of samples PyDex drew. The fallback's candidate sees failing tests in its prompt, so it lies outside $C_k(b)$ and is covered by Proposition 4.1 alone.

Proposition 4.4 (Shallowest composition). *If* Search *returns a candidate $c \in \Lambda_j$ for b, then no candidate in $\Lambda_0 \cup \cdots \cup \Lambda_{j-1}$ passes T^+; that is, c composes the fewest rule applications among the passing candidates of the enumerated layers.*

Proof. Search validates every candidate of Λ_j before it enumerates Λ_{j+1}, and stops at the first passing candidate. If c is returned from Λ_j, the layers $\Lambda_0, \ldots, \Lambda_{j-1}$ were therefore validated completely and held no passing candidate. Since each layer excludes the programs of earlier layers, an element of Λ_i is a program first reached by exactly i rule applications. □

Shallowest is not smallest, since one rule application may be a large edit; the enforced contract bounds how large. Write $|t|$ for the number of AST nodes in a fragment t.

Theorem 4.5 (Locality). *If* Search *returns $c \in \Lambda_j$ for b, then c differs from b at no more than j positions, and $\mathrm{TED}(b,c) \leq \sum_{i=1}^{j}(|t_i| + |t_i'|)$, where t_i and t_i' are the fragments the i-th rule application replaces and inserts.*

Proof. By induction on j. An element of Λ_j is obtained from one of Λ_{j-1} by one rule application, which the contract confines to one position, rewriting $C[t]$ to $C[t']$. Such a rewrite either lies apart from the earlier positions or absorbs those inside its hole, so it adds at most one position at which the program differs from b, and it costs at most $|t| + |t'|$ in tree edit distance, by deleting t's nodes and inserting t''s. Tree edit distance satisfies the triangle inequality, so the costs of the j applications add up. □

Hence $\mathrm{RPS}(b,c) \leq \sum_i(|t_i| + |t_i'|)/|b|$: a rule repair is as small as the fragments its rules rewrite, and Proposition 4.4 returns the fewest of them. The transplant obeys no such bound, which is why rule repairs are far smaller than transplanted ones (Section 5.4).

Proposition 4.6 (Monotone escalation). *For every b, if CoEP_L repairs b with f_L, then CoEP_{LF} repairs b with some f_{LF} such that $\mathrm{RPS}(b, f_{LF}) \leq \mathrm{RPS}(b, f_L)$.*

Proof. CoEP_{LF} computes the same local repair $f_{loc} = f_L$. If $\text{RPS}(b, f_L) \le \eta_{rps}$, it does not escalate and $f_{LF} = f_L$. Otherwise, f_{LF} is the arg min of $\text{RPS}(b, \cdot)$ over the candidates in $\{f_L, \text{LLM}(d, T, b, \pi(b))\}$ that pass T^+, a set that contains f_L by Proposition 4.1 and is therefore nonempty. □

Proposition 4.7 (Termination). *For every b, the Repair stage terminates.*

Proof. Each rule call is cut off after η_{alarm}, each validation runs under a per-test time limit, and the rule set and every layer are finite. Search validates at most η_{search} candidates. The transplant validates at most η_{cand} subsets in all, drawn from the η_{ref} reference programs, and Generate issues one query, retried at most five times under the client's request timeout. □

Because the search budget counts candidates rather than seconds, which repair is returned depends on machine load only through the per-test time limits; re-running every frozen tool on all 5,940 pairs reproduced every patch.

What is not guaranteed. CoEP is not complete: the transplant tries at most three hunks over at most 300 subsets and the search stops at its budget, so a repairable program may fall through to escalation or remain unrepaired. Nor is a repair correct beyond T^+: Proposition 4.1 bounds what CoEP accepts by T^+, not by the specification, Theorem 4.3 bounds departures from the class's labels under the generator's distribution, not under every input, and Section 5.6 measures the gap on held-out tests drawn from an independent generator.

5 Experiments

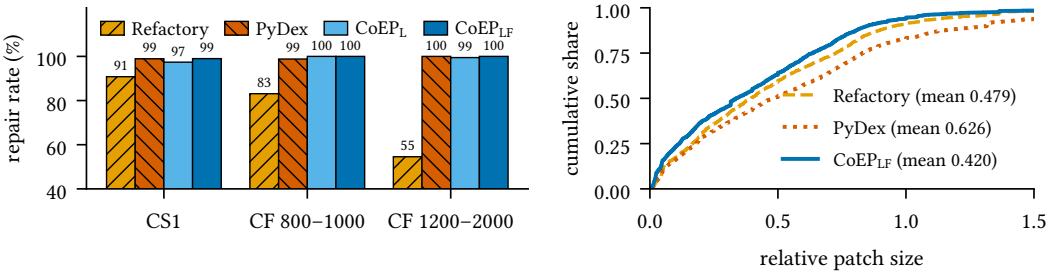
In this section, we evaluate CoEP to answer the following research questions:

(1) **Effectiveness:** How effectively does CoEP repair buggy programs compared to reference-guided and LLM-based repair, in terms of repair rate and patch size?
(2) **Efficiency:** How much model spend and turnaround does synthesizing a tool once per assignment save, compared to querying an LLM per submission?
(3) **Efficacy of Each Part:** What do the synthesized rules, the synthesized oracle, and the local-first architecture each contribute?
(4) **Generality:** Does a tool built from a class's earlier submissions repair its later ones, and how does its build scale with the class?
(5) **Correctness beyond the Tests:** Do the repairs remain correct on test cases that no arm searched against?

We implemented CoEP in Python on top of the standard `ast` module, and each synthesized tool is emitted as an ordinary Python module that runs offline. Tree-edit distance is computed with the Zhang–Shasha algorithm [45] and clustering with scikit-learn. All local computation ran on a Mac Studio (Apple M1 Ultra, 20 cores, 64 GB RAM, macOS 26.7).

5.1 Experimental Settings

Baselines. We compared CoEP with the two mainstream lines of repair for programming assignments: the reference-guided Refactory [11], the strongest such repairer that runs on our Python corpora, and the LLM-based PyDex [44]; Appendix A explains why other techniques could not serve. PyDex prompts a model with the buggy program and three similar (buggy, correct) exemplars retrieved from the class, samples ten candidates, filters them with T, and returns the survivor closest to the student's program; we re-implemented its few-shot semantic phase, billed all ten samples, and also priced it under prompt caching (PyDex_{PC}), with nine of its ten identical prompts at the providers' cache-read rate [2, 20]. We ran CoEP as CoEP_{L}, the frozen tool with no model at repair time, and as CoEP_{LF}, which adds the query on escalation. $\text{LLM}_{\text{Direct}}$ issues CoEP_{LF}'s query $\text{LLM}(d, T, b, \pi(b))$ for every buggy program, so it differs from CoEP_{LF} only in not reserving



(a) Repair rate by difficulty

(b) Patch size on the common pairs

Fig. 4. Repair rate and relative patch size of CoEP and the baselines.

the query for what the tool cannot serve. For the efficacy of each part, CoEP$_{LF}$ −Rules drops the builder's rules but keeps the always-on rules, the transplant, T^+, and the gated query, and CoEP$_{LF}$ −Amp accepts against T alone; CoEP$_L$−Syn is CoEP$_L$ without the builder's rules, and CoEP$_L$+Learn re-runs CoEP$_L$ with the learned rules. To separate the oracle from the repairer, we also ran PyDex accepting against T^+ (Section 5.6). Since the query does not depend on the tool and LLM$_{Direct}$ issued it for every program, CoEP$_{LF}$ and its variants take the answers to their escalations from LLM$_{Direct}$'s log, under CoEP$_{LF}$'s gate: the two arms then differ in the architecture alone, with the model's answers held fixed, and composing a live CoEP$_{LF}$ run this way reproduces its outcome on 99.93% of the pairs.

Benchmarks. We used 2,970 buggy programs from 11 assignments in two corpora, summarized in Table 2. The first five assignments (R1–R5) are those of a first-year university programming course, with every submission its students wrote, released by the authors of Refactory [11]. The other six (C1–C6) are Codeforces problems whose submissions the open-r1 release mirrors [24], one at each of the ratings 800, 1000, 1200, 1400, 1600, and 2000, so that difficulty spans five official rating bands from Newbie to Candidate Master [5]. We kept each corpus's own labels, so the denominator of every arm is the same 2,970 programs.

Other Settings. Every LLM arm ran on `gpt-5.4-mini` ($0.75/$4.50 per million input/output tokens) and `claude-sonnet-5` ($2/$10), each provider's cheapest tier at which prompt caching works on these prompts; each model served as CoEP's builder and fallback and as PyDex's repairer. On R1–R5, CoEP accepted repairs against T^+; on C1–C6, whose test cases are the judge's full official sets (22 to 56 per problem), $T^+ = T$. The *repair rate* (RR) counts a repair only when the fixed program passes all of T. The *relative patch size* (RPS, Section 4.5) is averaged over the repaired programs, and RR$_\tau$ is the share of programs repaired by a patch with RPS $\leq \tau$ that passes the held-out suite of Section 5.6 (on C1–C6, the official tests). The *turnaround* (ATT) is the mean wall-clock seconds per submission, local computation (ATT$_{cpu}$, for CoEP timed one program at a time on an otherwise idle machine over 20 programs per assignment, weighted by its size) plus model latency (ATT$_{api}$), and the *model cost* $C_{tot} = C_{build} + C_{api}$ is the US dollars spent over the benchmark, priced from exact token counts, with amplification in the build. For Refactory on R1–R5, we report the repair rates and patch sizes its authors published, and pair against the per-program results of the run they released, which reproduces those rates to within 0.05 points; on C1–C6, we ran the released tool. We report results over both builders but test each builder separately, with the exact McNemar test on paired repair outcomes and the Wilcoxon signed-rank test on patch sizes, at $p < 0.05$.

Table 2. Benchmarks and repair results. R1–R5: a first-year course; C1–C6: Codeforces problems (rating). R1 Sequential search; R2 Unique dates and months; R3 Duplicate elimination; R4 Sorting tuples; R5 Top-k elements; C1 Nearly Lucky Number (800); C2 cAPS lOCK (1000); C3 Planning The Expedition (1200); C4 Measuring Lengths in Baden (1400); C5 Jumping Jack (1600); C6 Digital Root (2000). Buggy, Correct, Tests: programs and instructor's test cases per assignment. Bold: the best value in each row and metric, ties included.

ID	Buggy	Correct	Tests	Repair rate (%)				Relative patch size			
				Refactory	PyDex	CoEP$_{\mathrm{L}}$	CoEP$_{\mathrm{LF}}$	Refactory	PyDex	CoEP$_{\mathrm{L}}$	CoEP$_{\mathrm{LF}}$
R1	575	768	11	99.0	**99.7**	**99.7**	**99.7**	0.390	0.428	0.290	**0.283**
R2	435	291	9	78.2	**97.8**	90.9	97.2	0.560	0.428	0.429	**0.424**
R3	308	546	6	97.4	**99.8**	98.4	98.7	0.340	0.423	**0.297**	0.298
R4	357	419	6	88.2	99.3	**100.0**	**100.0**	0.310	0.535	0.387	**0.292**
R5	108	418	5	88.0	95.4	**100.0**	**100.0**	0.320	0.244	**0.212**	**0.212**
C1	440	353	34	85.2	99.9	**100.0**	**100.0**	0.588	0.685	0.614	**0.525**
C2	560	140	56	81.4	97.9	**100.0**	**100.0**	0.724	1.167	0.770	**0.655**
C3	69	164	31	47.8	**100.0**	98.6	**100.0**	0.847	0.705	0.638	**0.582**
C4	38	37	22	92.1	**100.0**	**100.0**	**100.0**	0.605	0.643	0.560	**0.538**
C5	77	2	30	40.3	**100.0**	**100.0**	**100.0**	0.878	0.914	0.945	**0.830**
C6	3	2	40	**100.0**	**100.0**	**100.0**	**100.0**	2.248	1.975	**1.652**	**1.652**
Total	2,970	3,140	250	85.9	98.9	98.4	**99.4**	0.500	0.634	0.489	**0.436**

5.2 Effectiveness

Table 2 shows that CoEP$_{\mathrm{LF}}$ repaired as many programs as the strongest baseline while making the smallest patches. Overall, CoEP$_{\mathrm{LF}}$ repaired 99.4% of the 2,970 programs, against 85.9% for Refactory and 98.9% for PyDex, and reached 100% on eight of the eleven assignments. Its mean RPS of 0.436 is 31.2% smaller than PyDex's 0.634 and smaller than Refactory's 0.500, and on the 5,025 pairs all three arms repaired, it was 0.420 against 0.479 and 0.626 (Figure 4-(b), where a curve further to the left means smaller patches). PyDex's large patches are often rewrites: 17.8% and 17.1% of its repairs have RPS ≥ 1, against 6.0% and 6.8% of CoEP$_{\mathrm{LF}}$'s.

Notably, the gap to Refactory widens with difficulty. Grouped into the CS1 assignments, Codeforces 800–1000, and Codeforces 1200–2000, Refactory's repair rate falls from 90.8% to 83.1% and 54.5%, whereas CoEP$_{\mathrm{LF}}$ holds 99.0%, 100.0%, and 100.0% (Figure 4-(a)). Thin pools alone do not explain this collapse, since C5 and C6 give every arm only two reference programs and both CoEP arms repair them completely. The difference lies in the granularity of reuse: Refactory adopts divergent blocks, so a pool that solves the problem another way leaves nothing to adopt, whereas CoEP adopts line hunks after renaming.

The frozen tool alone, CoEP$_{\mathrm{L}}$, repaired 98.4% without a single model call, slightly fewer than PyDex's 98.9%; most of the gap lies on R2 and R3, where a candidate must also pass the 100 amplified test cases, and the escalation of CoEP$_{\mathrm{LF}}$ recovered most of it. Statistically, CoEP$_{\mathrm{LF}}$ repaired more programs than Refactory on both builders (exact McNemar, $p < 0.001$). Against PyDex, CoEP$_{\mathrm{LF}}$ repaired more programs on `gpt-5.4-mini` ($p = 0.029$) but not significantly on `claude-sonnet-5` ($p = 0.14$), and CoEP$_{\mathrm{L}}$'s difference was not significant ($p = 0.091$ and 0.075); both made smaller patches than both baselines on the common pairs (Wilcoxon signed-rank, $p < 0.01$ per builder). A tool synthesized once per assignment thus repairs as many programs as a model queried per submission, with patches a third smaller.

Table 3. Model cost over the benchmark ($) and turnaround per submission (s). ×: the arm's C_{tot} over CoEP_{LF}'s. N^*: submissions per assignment at which CoEP_{LF} breaks even with the arm. $\text{LLM}_{\text{Direct}}$'s turnaround is its model latency alone.

Arm	Model cost					Turnaround		
	C_{build}	C_{api}	C_{tot}	×	N^*	ATT_{cpu}	ATT_{api}	ATT_{tot}
`claude-sonnet-5`								
PyDex [44]	–	292.77	292.77	21.2	11	0.66	13.97	14.64
PyDex_{PC}	–	263.05	263.05	19.0	13	0.66	13.97	14.64
$\text{LLM}_{\text{Direct}}$	–	16.59	16.59	1.2	220	–	4.58	4.58
CoEP_{LF}	12.24	1.59	**13.83**	–	–	3.92	0.38	**4.29**
`gpt-5.4-mini`								
PyDex [44]	–	28.42	28.42	27.3	7	0.73	1.41	2.14
PyDex_{PC}	–	23.50	23.50	22.6	8	0.73	1.41	2.14
$\text{LLM}_{\text{Direct}}$	–	2.67	2.67	2.6	80	–	1.12	**1.12**
CoEP_{LF}	0.69	0.35	**1.04**	–	–	4.61	0.14	4.75

5.3 Efficiency

A per-submission arm pays for a query on every submission, whereas CoEP_{LF} pays its build once per assignment and queries only on escalation; the two break even at $N^* = \bar{C}_{build}/\Delta$ submissions per assignment, with $\bar{C}_{build}$ the mean build cost of an assignment and Δ the saving per submission. Figure 6 in Appendix B plots both costs on log scales, with circles at N^* and ticks at the sizes of our 11 assignments.

Table 3 shows that CoEP_{LF} spent $13.83 over the benchmark on `claude-sonnet-5` and $1.04 on `gpt-5.4-mini`, whereas PyDex spent $292.77 and $28.42, 21.2 and 27.3 times more. Since output tokens dominate the spend, PyDex_{PC} still cost 19.0 and 22.6 times more, and billing every input token at the cache-read rate would leave it 16.7 and 12.6 times more. An assignment repaid its build after $N^* = 11$ submissions against PyDex on `claude-sonnet-5` and 7 on `gpt-5.4-mini`, and ten of the eleven assignments exceeded these points, most by an order of magnitude. Even against $\text{LLM}_{\text{Direct}}$, which sends a single query without exemplars, CoEP_{LF} cost 1.2 and 2.6 times less; it breaks even later, at 220 and 80 submissions, but $\text{LLM}_{\text{Direct}}$ repaired only 82.4% of the programs. CoEP queried the model on 12.6% of the submissions (751 of 5,940 pairs). Regarding turnaround, CoEP_{LF} took 4.29 seconds per submission against PyDex's 14.64 on `claude-sonnet-5`, but 4.75 against 2.14 on `gpt-5.4-mini`, since a per-submission arm's turnaround is almost all model latency and CoEP's almost all local search; what CoEP pays instead is a build of 22.2 and 15.3 minutes per assignment, once before the first submission.

5.4 Efficacy of Each Part

Table 4 removes each synthesized part from CoEP_{LF}, and Figure 8 in Appendix D shows what repaired each program.

The architecture buys the saving. Without the builder's rules, CoEP's repair rate and held-out repair rate barely moved (2 of the 5,940 pairs changed), and its cost fell to $1.69 and $0.36, 173 and 79 times less than PyDex: the saving comes from the local-first architecture, a frozen search that repairs most submissions and escalates the rest. The reference transplant carries its breadth, producing 80.5% of CoEP_{L}'s 5,845 repairs at mean RPS 0.582. It needs no correct program with the buggy program's control flow, the requirement CLARA calls the only fundamental problem of its approach [9] and Sarfgen imposes exactly [36]: of the 5,904 pairs CoEP_{LF} repaired, 154 (2.6%)

Table 4. What each part of CoEP buys (`claude-sonnet-5` / `gpt-5.4-mini`). HELD-OUT: repair rate on T plus the held-out suite (R1–R5) or the official tests (C1–C6). $RR_{0.25}$: programs repaired with RPS ≤ 0.25 by a patch that passes them.

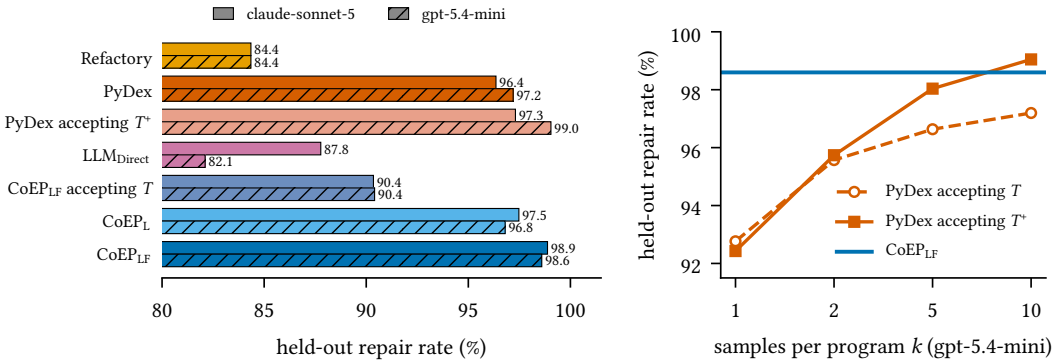
Arm	Rules	T^+	RR (%)	HELD-OUT (%)	RPS	$RR_{0.25}$ (%)	C_{tot} ($)
$CoEP_{LF}$	✓	✓	99.33 / 99.46	**99.33** / **99.16**	0.422 / 0.449	**42.1** / **38.1**	13.83 / 1.04
$CoEP_{LF}$ −Amp	✓	–	**99.63** / **99.66**	94.21 / 94.24	**0.406** / **0.431**	40.6 / 36.8	13.73 / 1.01
$CoEP_{LF}$ −Rules	–	✓	99.26 / 99.46	99.26 / **99.16**	0.448 / 0.458	36.4 / 36.3	1.69 / 0.36
−Rules −Amp	–	–	99.53 / **99.66**	93.97 / 94.11	0.431 / 0.440	34.7 / 34.9	**1.60** / **0.33**
PyDex			98.96 / 98.92	97.64 / 97.71	0.598 / 0.669	37.0 / 24.1	292.77 / 28.42

have a loop structure and 1,448 (24.5%) a nesting of loops and conditionals that no correct program of the class shares, and the tool repaired 1,258 of the latter without a query. Sending the same query for every program instead, LLM_{Direct} repaired only 82.4% (exact McNemar, $p < 0.001$ per builder), since a single query has no second candidate when its patch fails; on the 4,896 pairs both repaired, though, its patches were smaller (RPS 0.330 against 0.393), which credits the minimum-edit reference program $\pi(b)$ that both attach.

The oracle buys correctness. Without amplification, CoEP repaired slightly more programs on T but fewer that hold: its held-out repair rate fell from 99.3% and 99.2% to 94.2% on both builders (exact McNemar, $p < 0.001$ per builder; 155 and 149 programs against 3 and 3), while the build it saved was the generator's price, $0.02 and $0.01. Section 5.6 shows that this is the interaction Theorem 4.3 predicts.

The rules buy small patches. The synthesized rules matter for patch size and thin pools rather than coverage. With them, CoEP repaired 5.7 and 1.8 points more programs with a held-out-passing patch of RPS at most 0.25 (174 and 54 programs against 5 and 0; exact McNemar, $p < 0.001$ per builder), and on the programs both arms repaired, its patches were smaller (RPS 0.423 against 0.448 and 0.449 against 0.458; Wilcoxon, $p < 0.001$). Synthesized rules took part in 836 of the 1,142 rule repairs, which make the smallest patches in the system at mean RPS 0.109 against 0.582 for the transplant, as Theorem 4.5 leads one to expect, and 405 of those repairs (35.5%) composed two to five rule applications (Figure 8-(b)), reached only after every shallower composition had failed (Proposition 4.4). The coverage gain is small because the transplant repairs nearly everything a rule repairs when a matching pool exists (99.2%), When the pool is thin, as on a new assignment, the rules keep repairs local and small: with the tools of Section 5.5 built from a class's first 10% and a pool of only its first one, two, or five correct programs, on the 1,630 programs per builder that no builder saw, they averted 69 and 18 escalations against none at one program and raised $RR_{0.25}$ by 2.1 to 4.8 points (exact McNemar, $p < 0.001$), while coverage stayed at 99.1 to 99.8% either way; with no correct program at all, they raised $CoEP_L$'s repair rate on 146 unseen programs from 8.2% to 28.8%. The rules also name what they repair: three raters judged 191 rules against their code and a repair each produced (Fleiss' $\kappa = 0.890$), and 99.2% of the rule-produced repairs carry a name consistent with what the rule did.

Learn. The Learn stage fires only where the tool fails: $CoEP_L$ failed on 95 pairs, the fallback repaired 59, and the Learn stage generalized these repairs into 11 rules for $0.89, not included in Table 3, with which $CoEP_L$+Learn recovered 15 programs without a regression (14 on `claude-sonnet-5`, exact McNemar $p < 0.001$; one on `gpt-5.4-mini`), against 32 before the contract was enforced, since an LLM patch often edits several positions. This is reuse within an assignment, not transfer across assignments. The 36 pairs $CoEP_{LF}$ left unrepaired all reached the fallback: 18 are repaired when accepting against T alone, and for the other 18, all on R2, no candidate passed T.



(a) Held-out repair rate of each arm (b) PyDex's samples against its oracle

Fig. 5. Held-out repair rate on R1–R5 of each arm and of PyDex against its number of samples. Accepting T or T^+: accepting a candidate against T alone or against CoEP's amplified suite.

5.5 Generality

We examined whether a tool built from a class's earlier submissions repairs its later ones. We ordered the submissions of C1–C5 by id, which follows submission time, and those of CS1, which records neither time nor student, in three seeded random orders (C6 has five submissions). We built a tool from the first $f \in \{10, 25, 50, 75\}\%$ of each order, its buggy programs feeding Select and its correct programs forming R, and repaired the buggy programs of the last quarter, 439 per order on CS1 and 313 on Codeforces, also with the tools of Section 5.2, built from the whole class ("all" in Figure 7-(a), Appendix C).

A tool built from the first 10% of a class repaired its later submissions about as well as one built from the whole class (Figure 7-(a)): CoEP_{LF} repaired 99.3% on CS1 (against 99.1% for the whole-class tool) and 98.9% on Codeforces (against 100.0%), rising to 99.5% from half the class. The frozen tool alone needed more history on Codeforces, where a thin pool offers fewer reference programs to transplant from (93.9% from 10% and 98.4% from 50%), and the fallback recovered most of the difference. From 10% to 75% of CS1, the worked pairs grew 9.2-fold but the build cost only 1.9-fold (Appendix C), so an instructor facing a new assignment can build a tool from its first few dozen submissions and rebuild it as the term proceeds.

5.6 Correctness beyond the Tests

R1–R5 ship 7.4 instructor test cases per assignment, and in Section 5.2 every arm both searched against them and was judged by them. To check whether the repairs remain correct beyond them, we built a held-out suite per assignment that no arm searched against: 100 inputs drawn with a different seed from a generator we wrote by hand, independently of the builders' generators, each labelled by the majority output of a different sample of 40 correct programs and the reference solution at 90% agreement, and none in T^+. We then re-ran every arm's repairs on T and the held-out suite; C1–C6 already use the judge's full official test sets.

Figure 5-(a) shows that CoEP's repairs held more often than those of the baselines run with their own acceptance. Of the programs CoEP_{LF} repaired, 100.0% and 99.5% also passed the held-out suite, against 97.7% and 98.0% for PyDex, so on the held-out suite CoEP_{LF} repaired 98.9% and 98.6% of the programs against PyDex's 96.4% and 97.2% (exact McNemar, $p < 0.001$ and $p = 0.005$). Accepting against T alone, the same tool repaired 90.4% on both builders: amplification traded 0.5 and 0.3 points of repair rate on T for 8.5 and 8.2 points on the held-out suite. Every held-out label equals the

reference solution's output, and a boundary suite we wrote by hand from each problem statement, 7 to 12 empty, singleton, extreme, and tied inputs per assignment labelled by the reference solution and none in T^+, gives the same verdict: CoEP_{LF} passed it on 98.5% and 99.1% of the programs, PyDex on 94.7% and 97.1% ($p < 0.001$ per builder), and CoEP_{LF} accepting T alone on 87.9% and 88.0%.

The oracle, not the search, buys correctness, and it transfers. Given the same T^+, PyDex's held-out repair rate rose by 0.9 and 1.8 points, to 97.3% and 99.0%; with equal oracles, CoEP remained ahead on `claude-sonnet-5` (98.9% against 97.3%, exact McNemar $p < 0.001$) and tied on `gpt-5.4-mini` (98.6% against 99.0%, $p = 0.27$). CoEP kept its patches 21% and 35% smaller on the programs both repaired (RPS 0.307 against 0.389 and 0.324 against 0.497; Wilcoxon, $p < 0.001$) at 19 and 30 times lower cost; sampling only until a candidate passes T^+ cuts PyDex's cost on `gpt-5.4-mini` to $2.28, still 3.9 times CoEP's $0.59, with patches at RPS 0.596. We re-queried PyDex on `gpt-5.4-mini` keeping all ten candidates, and on `claude-sonnet-5` for the 69 programs whose pick fails T^+, the only ones a stricter oracle can change.

Figure 5-(b) shows the breadth effect Theorem 4.3 predicts, within one repairer: as PyDex draws more samples, accepting against T^+ instead of T costs 0.4 points of held-out repair rate at $k = 1$ (92.4% against 92.8%) and gains 1.8 at $k = 10$ (99.0% against 97.3%). With a single candidate, a stricter oracle can only reject, and $\text{LLM}_{\text{Direct}}$, which sends one query, indeed lost held-out repairs under T^+ (87.8% and 82.1%, against 90.2% and 84.0% under T), whereas a search, CoEP's or PyDex's, finds another candidate that passes. The rest are explicable: `gpt-5.4-mini`'s R3 generator drew values only from -2 to 3, missing nine patches that fail on larger ones.

5.7 Threats to Validity

(1) **Class splits:** The CS1 splits of Section 5.5 are random orders; a split by term may yield lower rates. (2) **Build and evaluation overlap:** The tools of Sections 5.2–5.4 and 5.6 were built from the class they repair, with 1,524 of the 2,970 buggy programs among the worked pairs; Section 5.5 evaluates on submissions the builder never saw. (3) **Test adequacy:** A repair counts when it passes T. On R1–R5, CoEP also requires T^+ and Section 5.6 checks held-out tests, but both suites are labelled by the class's correct programs, so a behavior most of the class gets wrong would go undetected (Proposition 4.2), although every label also equals the reference solution's output and the boundary suite checks edge cases without the class; on C1–C6, we rely on the judge's official tests. (4) **Builders:** Both builders are closed commodity models, and a rebuild writes other rules; rebuilding every tool and generator twice more ($31) moved CoEP_{LF}'s repair rate by at most 0.07 points, its held-out repair rate by 0.3, its RPS by 0.009, and its $\text{RR}_{0.25}$ by 2.5, and kept its cost 18 and 23 times below PyDex's. We release the 22 tools and 10 input generators, which run offline. (5) **Benchmarks:** Our 11 assignments are in Python, and C6 contributes three buggy programs; the results may not transfer to other languages or large projects.

6 Related Work

Program Repair for Programming Assignments. Singh et al. [31] repair an attempt with an instructor-written *error model*, a set of rewrite rules whose combinations a Sketch-based search explores for a minimal correction; CoEP synthesizes that error model per assignment and searches it by enumeration validated against T^+. Reference-guided techniques reuse the class's correct programs: CLARA [9] and Sarfgen [36] align a buggy program with a correct one of the same loop structure or sequence of control statements, which a quarter of our repairs lack (Section 5.4), Verifix [1] backs the alignment with formal verification, Refactory [11] refactors the pool to widen what aligns, and MENTORED [4] recombines correct and incorrect programs by genetic programming. LLM-based techniques query a model per submission, with retrieved exemplars [44],

tutoring [40], fine-tuning [14, 41], fault localization [6, 22, 23], or other students' fixes [7]. Unlike these techniques, CoEP spends its model effort on a tool built once per assignment; any of them can serve as the GENERATE function behind it.

Clustering and Transforming Submissions. OverCode [8], CLARA [9], SemCluster [25], and InvAASTCluster [21] group programs whose normalized statements, control flow and variable traces, per-path semantic features, or invariants and anonymized ASTs coincide, to choose the reference program a repairer aligns with. CoEP instead clusters buggy programs by the edit that repairs them, since its clusters decide which repairs a builder generalizes. Head et al. [10] group incorrect submissions by the transformations that fix them, so that a teacher writes feedback once per group. Like CoEP, Refazer [27] induces assignment-specific rewrite rules from (before, after) pairs, by programming-by-example in a domain-specific language from students' own fixes; CoEP's builder instead sees the problem description and tests with each pair, so a rule can encode the misconception rather than the syntax of one fix, and CoEP synthesizes the oracle that validates the rules as well. Template-based repair [13, 15] and Reyes et al. [26] apply mined or LLM-generated AST transformations to software projects rather than to a class.

LLMs as Tool Makers. In a line of research LLMs write reusable tools instead of solving each task directly: LATM [3], CRAFT [43], TroVE [37], ReGAL [34], Voyager [35], FunSearch [28], ReaComp [19]. Most of them keep a model in the loop that calls the tools at inference, whereas CoEP deploys its tool with no model behind it, and program repair adds an executable oracle for the tool itself. Closer to programming languages, type-constrained decoding guarantees well-typed LLM output by checking each prefix against the type system [18]; CoEP instead executes everything the LLM writes against T^+ (Proposition 4.1), an oracle it also synthesizes.

7 Conclusion

Recently, LLM-based program repair has substantially improved the share of student submissions that can be repaired automatically. However, these techniques query the model once per buggy submission, so a class pays again for every recurring repair, and they accept any patch that passes the instructor's few tests. This paper presents CoEP, which has an LLM synthesize, once per assignment, the assignment-specific parts of a repair tool, its rewrite rules and the input generator behind its oracle, rather than a patch for every submission. CoEP selects worked pairs by clustering the class's edit scripts, synthesizes misconception-named rewrite rules, amplifies the test cases by differential testing against the class's correct programs, repairs every submission locally, and learns new rules from the few it escalates. Experimental results demonstrate that CoEP matches per-submission LLM repair in repair rate at a fraction of the model spend, with the saving owed to its local-first architecture, correctness beyond the instructor's tests to its synthesized oracle, and smaller patches to its synthesized rules. We expect the principle that a broader search needs a stronger oracle to carry over to LLM-synthesized tools beyond the repair of assignments.

AI Use

LLMs are part of the technique this paper studies: `claude-sonnet-5` and `gpt-5.4-mini` synthesized the rules and input generators and served as the fallback, the Learn stage, and the PyDex baseline (Sections 4–5), and every output they produced in the experiments was validated by execution. The authors also used an AI assistant to edit the manuscript and to write analysis scripts; every number was recomputed from the released logs.

Data-Availability Statement

CoEP's implementation, the benchmark with its held-out and amplified test suites, the synthesized tools, every prompt, and the per-submission logs, which reproduce every number in this paper without model access, are available at https://anonymous.4open.science/r/coep/.

References

[1] Umair Z Ahmed, Zhiyu Fan, Jooyong Yi, Omar I Al-Bataineh, and Abhik Roychoudhury. 2022. Verifix: Verified repair of programming assignments. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 31, 4 (2022), 1–31.

[2] Anthropic. 2025. Prompt Caching — Claude API Documentation. https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching. Cache reads are billed at 0.1x the base input-token price; accessed July 2026.

[3] Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. 2024. Large language models as tool makers. In *International Conference on Learning Representations*, Vol. 2024. 54067–54089.

[4] Dongwook Choi and Eunseok Lee. 2025. Automated Feedback Generation for Programming Assignments Through Diversification. In *2025 IEEE/ACM 37th International Conference on Software Engineering Education and Training (CSEE&T)*. IEEE, 230–241.

[5] Codeforces. 2024. Codeforces Rating System. https://codeforces.com/blog/entry/68288. Rating bands: Newbie, Pupil, Specialist, Expert, Candidate Master; accessed July 2026.

[6] Zhenlong Dai, Bingrui Chen, Zhuoluo Zhao, Xiu Tang, Sai Wu, Chang Yao, Zhipeng Gao, and Jingyuan Chen. 2025. Less is more: Adaptive program repair with bug localization and preference learning. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 39. 128–136.

[7] Zhenlong Dai, Zhuoluo Zhao, Hengning Wang, Xiu Tang, Sai Wu, Chang Yao, Zhipeng Gao, and Jingyuan Chen. 2026. Learner-Tailored Program Repair: A Solution Generator with Iterative Edit-Driven Retrieval Enhancement. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 30467–30475.

[8] Elena L Glassman, Jeremy Scott, Rishabh Singh, Philip J Guo, and Robert C Miller. 2015. OverCode: Visualizing variation in student solutions to programming problems at scale. *ACM Transactions on Computer-Human Interaction (TOCHI)* 22, 2 (2015), 1–35.

[9] Sumit Gulwani, Ivan Radiček, and Florian Zuleger. 2018. Automated clustering and program repair for introductory programming assignments. *ACM SIGPLAN Notices* 53, 4 (2018), 465–480.

[10] Andrew Head, Elena Glassman, Gustavo Soares, Ryo Suzuki, Lucas Figueredo, Loris D'Antoni, and Björn Hartmann. 2017. Writing reusable code feedback at scale with mixed-initiative program synthesis. In *Proceedings of the Fourth (2017) ACM Conference on Learning@ Scale*. 89–98.

[11] Yang Hu, Umair Z Ahmed, Sergey Mechtaev, Ben Leong, and Abhik Roychoudhury. 2019. Re-factoring based program repair applied to programming assignments. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 388–398.

[12] René Just, Darioush Jalali, Laura Inozemtseva, Michael D Ernst, Reid Holmes, and Gordon Fraser. 2014. Are mutants a valid substitute for real faults in software testing?. In *Proceedings of the 22nd ACM SIGSOFT international symposium on foundations of software engineering*. 654–665.

[13] Dongsun Kim, Jaechang Nam, Jaewoo Song, and Sunghun Kim. 2013. Automatic patch generation learned from human-written patches. In *2013 35th international conference on software engineering (ICSE)*. IEEE, 802–811.

[14] Fang Liu, Zhenwei Liu, Qianhui Zhao, Jing Jiang, Li Zhang, Zian Sun, Ge Li, Zhongqi Li, and Yuchi Ma. 2024. Fastfixer: An efficient and effective approach for repairing programming assignments. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 669–680.

[15] Kui Liu, Anil Koyuncu, Dongsun Kim, and Tegawendé F Bissyandé. 2019. TBar: Revisiting template-based automated program repair. In *Proceedings of the 28th ACM SIGSOFT international symposium on software testing and analysis*. 31–42.

[16] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. 2023. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems* 36 (2023), 46534–46594.

[17] William M McKeeman. 1998. Differential testing for software. *Digital Technical Journal* 10, 1 (1998), 100–107.

[18] Niels Mündler, Jingxuan He, Hao Wang, Koushik Sen, Dawn Song, and Martin Vechev. 2025. Type-constrained code generation with language models. *Proceedings of the ACM on Programming Languages* 9, PLDI (2025), 601–626.

[19] Atharva Naik, Yash Mathur, Carolyn Rose, David Mortensen, et al. 2026. ReaComp: Compiling LLM Reasoning into Symbolic Solvers for Efficient Program Synthesis. *arXiv preprint arXiv:2605.05485* (2026).

[20] OpenAI. 2024. Prompt Caching — OpenAI API Documentation. https://platform.openai.com/docs/guides/prompt-caching. Automatic prefix caching with discounted cached input tokens; see also https://openai.com/index/api-prompt-caching/; accessed July 2026.

[21] Pedro Orvalho, Mikoláš Janota, and Vasco Manquinho. 2025. InvAASTCluster: on applying invariant-Based program clustering to introductory programming assignments. *Journal of Systems and Software* 230 (2025), 112481.

[22] Pedro Orvalho, Mikoláš Janota, and Vasco Manquinho. 2025. MENTOR: Fixing introductory programming assignments with formula-based fault localization and LLM-driven program repair. *Journal of Systems and Software* (2025), 112690.

[23] Pedro Orvalho, Mikoláš Janota, and Vasco M Manquinho. 2025. Counterexample guided program repair using zero-shot learning and maxsat-based fault localization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 649–657.

[24] Guilherme Penedo, Hynek Kydlíček, Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Leandro von Werra, and Thomas Wolf. 2025. Codeforces. https://huggingface.co/datasets/open-r1/codeforces.

[25] David M Perry, Dohyeong Kim, Roopsha Samanta, and Xiangyu Zhang. 2019. SemCluster: clustering of imperative programming assignments based on quantitative semantic features. In *Proceedings of the 40th acm sigplan conference on programming language design and implementation*. 860–873.

[26] Frank Reyes, Benoit Baudry, and Martin Monperrus. 2026. Agentic Generation of AST Transformation Rules for Fixing Breaking Updates. *arXiv preprint arXiv:2606.24446* (2026).

[27] Reudismam Rolim, Gustavo Soares, Loris D'Antoni, Oleksandr Polozov, Sumit Gulwani, Rohit Gheyi, Ryo Suzuki, and Björn Hartmann. 2017. Learning syntactic program transformations from examples. In *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. IEEE, 404–415.

[28] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M Pawan Kumar, Emilien Dupont, Francisco JR Ruiz, Jordan S Ellenberg, Pengming Wang, Omar Fawzi, et al. 2024. Mathematical discoveries from program search with large language models. *Nature* 625, 7995 (2024), 468–475.

[29] Peter J Rousseeuw. 1987. Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. *Journal of computational and applied mathematics* 20 (1987), 53–65.

[30] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems* 36 (2023), 8634–8652.

[31] Rishabh Singh, Sumit Gulwani, and Armando Solar-Lezama. 2013. Automated feedback generation for introductory programming assignments. In *Proceedings of the 34th ACM SIGPLAN conference on Programming language design and implementation*. 15–26.

[32] Edward K Smith, Earl T Barr, Claire Le Goues, and Yuriy Brun. 2015. Is the cure worse than the disease? overfitting in automated program repair. In *Proceedings of the 2015 10th joint meeting on foundations of software engineering*. 532–543.

[33] Armando Solar-Lezama, Liviu Tancau, Rastislav Bodik, Sanjit Seshia, and Vijay Saraswat. 2006. Combinatorial sketching for finite programs. In *Proceedings of the 12th international conference on Architectural support for programming languages and operating systems*. 404–415.

[34] Elias Stengel-Eskin, Archiki Prasad, and Mohit Bansal. 2024. Regal: Refactoring programs to discover generalizable abstractions. *arXiv preprint arXiv:2401.16467* (2024).

[35] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291* (2023).

[36] Ke Wang, Rishabh Singh, and Zhendong Su. 2018. Search, align, and repair: data-driven feedback generation for introductory programming exercises. In *Proceedings of the 39th ACM SIGPLAN conference on programming language design and implementation*. 481–495.

[37] Zhiruo Wang, Daniel Fried, and Graham Neubig. 2024. Trove: Inducing verifiable and efficient toolboxes for solving programmatic tasks. *arXiv preprint arXiv:2401.12869* (2024).

[38] Chunqiu Steven Xia and Lingming Zhang. 2024. Automated program repair via conversation: Fixing 162 out of 337 bugs for $0.42 each using chatgpt. In *Proceedings of the 33rd ACM SIGSOFT international symposium on software testing and analysis*. 819–831.

[39] Qi Xin and Steven P Reiss. 2017. Identifying test-suite-overfitted patches through test case generation. In *Proceedings of the 26th ACM SIGSOFT international symposium on software testing and analysis*. 226–236.

[40] Boyang Yang, Haoye Tian, Weiguo Pian, Haoran Yu, Haitao Wang, Jacques Klein, Tegawendé F Bissyandé, and Shunfu Jin. 2024. Cref: An llm-based conversational software repair framework for programming tutors. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 882–894.

[41] Boyang Yang, Haoye Tian, Jiadong Ren, Hongyu Zhang, Jacques Klein, Tegawendé F Bissyandé, Claire Le Goues, and Shunfu Jin. 2026. Morepair: Teaching llms to repair code via multi-objective fine-tuning. *ACM Transactions on Software Engineering and Methodology* 35, 2 (2026), 1–38.

[42] Jooyong Yi, Umair Z Ahmed, Amey Karkare, Shin Hwei Tan, and Abhik Roychoudhury. 2017. A feasibility study of using automated program repair for introductory programming assignments. In *Proceedings of the 2017 11th joint meeting on foundations of software engineering*. 740–751.

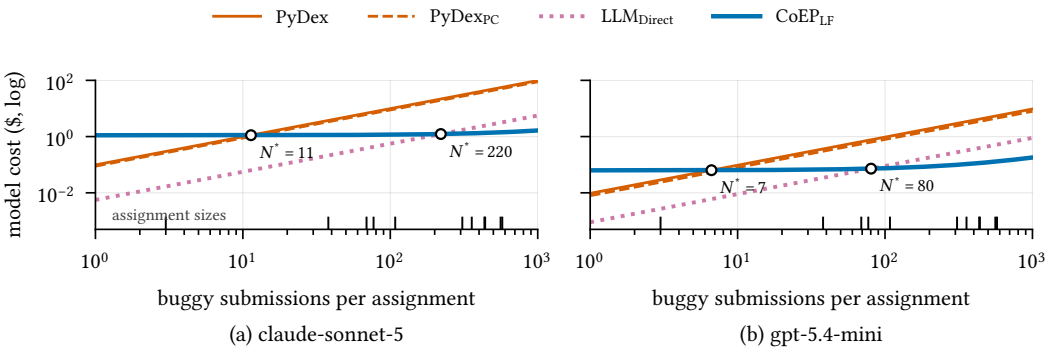


Fig. 6. Model cost of one assignment against its number of buggy submissions.

[43] Lifan Yuan, Yangyi Chen, Xingyao Wang, Yi Fung, Hao Peng, and Heng Ji. 2024. Craft: Customizing llms by creating and retrieving from specialized toolsets. In *International Conference on Learning Representations*, Vol. 2024. 40097–40125.

[44] Jialu Zhang, José Pablo Cambronero, Sumit Gulwani, Vu Le, Ruzica Piskac, Gustavo Soares, and Gust Verbruggen. 2024. Pydex: Repairing bugs in introductory python assignments using llms. *Proceedings of the ACM on Programming Languages* 8, OOPSLA1 (2024), 1100–1124.

[45] Kaizhong Zhang and Dennis Shasha. 1989. Simple fast algorithms for the editing distance between trees and related problems. *SIAM journal on computing* 18, 6 (1989), 1245–1262.

A Selection of Baselines

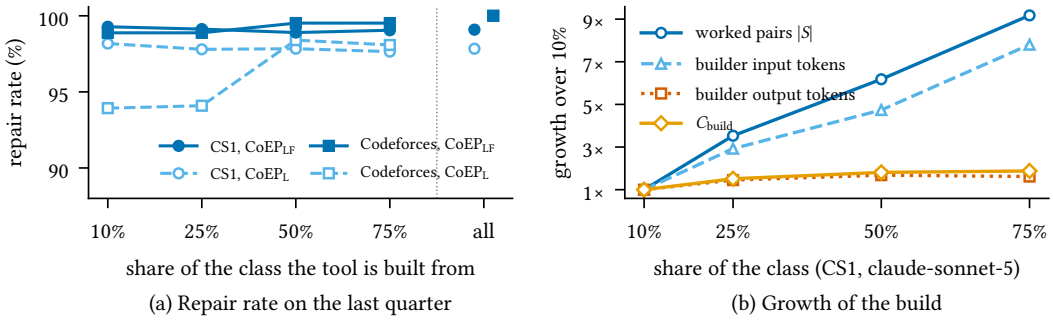
We selected the baselines of Section 5.1 by four criteria: a technique must (1) produce repairs rather than clusters or tools for other tasks, (2) repair Python programs, (3) require no input our benchmark lacks, such as the link between a student's successive attempts, a tutor in the loop, or a model trained for the task, and (4) represent its line of repair on the CS1 corpus. Only Refactory and PyDex satisfy all four. Clustering, teacher-in-the-loop feedback, and tool-making approaches [3, 8, 10, 21, 25, 34, 37, 43] fail (1); Verifix [1], SemCluster, InvAASTCluster, and the counterexample-guided repairers [22, 23] (C), Sarfgen [36] (C#), and template- and transformation-based repair [13, 15, 26] (software projects) fail (2); and Refazer [27] and LSGen [7], which learn from or retrieve a student's successive attempts, Singh et al. [31], which needs an instructor-written error model per assignment, CREF [40], which keeps a tutor in the loop, and FastFixer [14], AdaPatcher [6], and MORepair [41], which fine-tune a model, fail (3). Of the remaining techniques, CLARA [9] and MENTORED [4] reuse the class's submissions as Refactory does, and we take Refactory, whose authors released the CS1 corpus and evaluated it against CLARA there [11], as the representative of that line; PyDex, unlike general-purpose repairers such as ChatRepair [38], is designed for introductory Python assignments and publishes its prompts in full [44].

B Break-Even Curves

C Building on Earlier Submissions

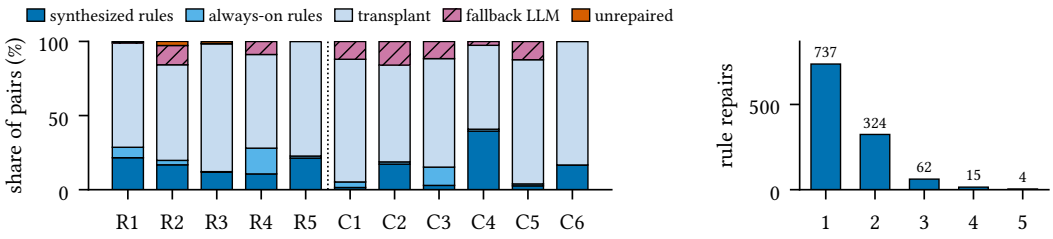
Figure 7 shows the tools of Section 5.5. Their build cost grew far more slowly than the class (Figure 7-(b)): from 10% to 75% of CS1, the worked pairs grew 9.2-fold, from 15.6 to 143.1 per assignment, whereas the build cost on `claude-sonnet-5` grew 1.9-fold, from $0.88 to $1.66, since the builder's input grew 7.8-fold but its output, the module of rules that dominates the price, only 1.6-fold: most new pairs fall into misconceptions the tool has already named.

D Source of Each Repair



(a) Repair rate on the last quarter (b) Growth of the build

Fig. 7. Repair rate and build growth of tools built from the first f of a class.



(a) Source of each repair (b) Composition depth

Fig. 8. Source of each repair and depth of the rule compositions.